

arr	i	arr[i]	y	x
[3,4,-8,10,-6,8]			0	3
	0	3	3	3
	1	4	7	7
	2	-8	-1	
			0	
	3	10	10	10
	4	-6	4	10
	5	8	12	12

```

public static int what(int[] arr) {
    int y = 0;
    int x = arr[0];

    for (int i = 0; i < arr.length; i++) {
        y += arr[i];
        x = Math.max(x, y);
        y = Math.max(y, 0);
    }

    return x;
}

```

ממשו את הפעולה arrange בשפת Java או בשפת C# :

Java – public static void arrange (int[] arr)

C# – public static void Arrange (int[] arr)

במערך arr שמתקבל יש מספרים חיוביים ומספרים שליליים.

הפעולה מעבירה לתחילת המערך arr את כל המספרים החיוביים (כולל 0), וממקמת אחריהם את כל המספרים השליליים.

הסדר של המספרים החיוביים (בינם לבין עצמם) יישאר באותו הסדר שבו הם היו לפני הפעולה, וגם הסדר של המספרים

השליליים (בינם לבין עצמם) יישאר באותו הסדר שבו הם היו לפני הפעולה.

דוגמה:

בעבור המערך arr שלפניכם:

	0	1	2	3	4	5	6	7
arr	7	-2	-5	61	4	-1	0	33

המערך arr ייראה כך בסיום הפעולה:

	0	1	2	3	4	5	6	7
arr	7	61	4	0	33	-2	-5	-1

פונקצית עזר: מקבלת מערך ומחזירה העתק שלו //

```

1 public static int[] copy(int[] arr) {
2     int[] clone = new int[arr.length];
3     for (int i = 0; i < arr.length; i++) {
4         clone[i] = arr[i];
5     }
6     return clone;
7 }

```

הפעולה מעבירה לתחילת המערך arr את כל המספרים החיוביים (כולל 0), וממקמת אחריהם את כל המספרים השליליים.

הסדר של המספרים החיוביים (בינם לבין עצמם) יישאר באותו הסדר שבו הם היו לפני הפעולה, וגם הסדר של המספרים

השליליים (בינם לבין עצמם) יישאר באותו הסדר שבו הם היו לפני הפעולה.

```

1 public static void arrange(int[] arr) {
2     int[] tmp = copy(arr);
3     int next = 0;
4     for (int i = 0; i < tmp.length; i++) {
5         if (tmp[i] >= 0) {
6             arr[next] = tmp[i];
7             next++;
8         }
9     }
10    for (int i = 0; i < tmp.length; i++) {
11        if (tmp[i] < 0) {
12            arr[next] = tmp[i];
13            next++;
14        }
15    }
16 }

```

מספר חיובי – num ייקרא "מספר נרקיסיסטי" אם מתקיים התנאי שלפניכם:

מעלים כל אחת מן הספרות של num בחזקת כמות הספרות שיש ב"num (אורך המספר num).

מחברים את כל התוצאות, והסכום שמתקבל שווה ל"num .

דוגמה ל"מספר נרקיסיסטי": המספר 407, כי כאשר מעלים כל אחת מן הספרות שלו בחזקת 3 (שהיא כמות הספרות

במספר) מקבלים 407 : $4^3 + 0^3 + 7^3 = 64 + 0 + 343 = 407$

```
1 // מקבלת מספר ומחזירה כמות הסיפרות בו
2 public static int length(int num) {
3     int len = 0;
4     while (num > 0) {
5         len++;
6         num /= 10;
7     }
8     return len;
9 }
10
11 print(length(1234) == Integer.toString(1234).length());
12 print(length(1234) == String.valueOf(1234).length());
13
14 // מחזירה מזלה של מספר
15 public static int pow(int num, int p) {
16     int mul = 1;
17     while (p > 0) {
18         mul *= num;
19         p--;
20     }
21     return mul;
22 }
23
24 print(pow(2, 5) == Math.pow(2, 5));
```

כתבו פעולה חיצונית ששמה isNarc בשפת Java או IsNarc בשפת C#, המקבלת מספר מטיפוס שלם – num הגדול מ־0. הפעולה מחזירה true אם המספר נרקסיסיטי, אחרת היא מחזירה false.

```
1 public static boolean isNarc(int num) {
2     int sum = 0;
3     int len = Integer.toString(num).length();
4     int n = num;
5     while (n > 0) {
6         sum += Math.pow(n % 10, len);
7         n /= 10;
8     }
9     return num == sum;
10 }
```

כתבו פעולה חיצונית ששמה theNarc בשפת Java או TheNarc בשפת C#, המקבלת מספר מטיפוס שלם – n הגדול מ־0. הפעולה מדפיסה את כל המספרים הנרקסיסיטיים שיש מ־1 ועד n (כולל).
הערך: יש להשתמש בפעולה שכתבתם בסעיף א.

```
1 public static void theNarc(int n) {
2     for (int i = 1; i <= n; i++) {
3         if (isNarc(i)) {
4             System.out.println(i);
5         }
6     }
7 }
```

למועצה האזורית "עוז" משתייכים כמה יישובים. המועצה מספקת לילדים הגרים ביישובים אלה שירותי הסעה מן הבית אל בית הספר ובחזרה. הסעה מן הבית אל בית הספר נקראת "הסעה הלון" והסעה בחזרה מבית הספר אל הבית נקראת "הסעה חזור".

נתונה המחלקה **Transport** – הסעה, ולה ארבע תכונות:

- name – שם היישוב שבעבורו מתבצעת ההסעה, מטיפוס מחרוזת
- toSchool – סוג ההסעה. בעבור "הסעה הלון" ערך התכונה הוא true, ובעבור "הסעה חזור" הערך הוא false.
- num – מספר התלמידים בהסעה – מספר מטיפוס שלם בין 1 ל־50 (כולל)
- day – מספר מטיפוס שלם בין 1 ל־6 (כולל), המייצג את היום בשבוע שבו ההסעה מתבצעת (ראשון–שישי בהתאמה)

הניחו שקיימות פעולות get/Get לתכונות המחלקה.

(1) כתבו פעולה בונה במחלקה **Transport**, המקבלת את שם היישוב שבעבורו ההסעה – name, מספר התלמידים בהסעה – num, והיום בשבוע שבו ההסעה מתבצעת – day. הפעולה תאתחל את תכונות המחלקה. בסוג ההסעה (toSchool) הפעולה תיצור "הסעה הלוך". הניחו שהפרמטרים שמתקבלים תקינים.

(2) כתבו פעולה בונה במחלקה **Transport** המקבלת רק את מספר התלמידים בהסעה – num. הפעולה תאתחל את תכונות המחלקה כך שתקבל "הסעה חזור" ביישוב Aviv ביום רביעי עם num תלמידים. הערה: אם פרמטר מספר התלמידים (num) אינו תקין, התכונה תקבל 1.

```
class Transport {
    private String name;
    private boolean toSchool;
    private int num;
    private int day;

    public Transport(String name, int num, int day) {
        this.name = name;
        this.toSchool = true;
        this.num = num;
        this.day = day;
    }

    public Transport(int num) {
        if (num < 1 || num > 50) {
            num = 1;
        }
        this.name = "Aviv";
        this.toSchool = false;
        this.num = num;
        this.day = 4;
    }
}
```

נתון מערך – arr, מטיפוס **Transport**, המכיל את ההסעות שהתבצעו במהלך השבוע. המערך אינו ממוין לפי סדר כלשהו, ואין בו ערכי null.

(1) כתבו פעולה חיצונית ששמה dayReport בשפת Java או בשפת C#, המקבלת את המערך arr, מספר מטיפוס שלם – day המציין יום בשבוע, ופרמטר בוליאני – forward. אם forward הוא true, הפעולה תחזיר את סך מספר התלמידים שנסעו ב"הסעות הלוך" ביום – day. אחרת (אם forward הוא false), הפעולה תחזיר את סך מספר התלמידים שנסעו ב"הסעות חזור" ביום – day. הניחו שהפרמטר day שמתקבל תקין (בין 1 ל-6).

```
1 Transport[] arr = new Transport[] {
2     new Transport("Aviv", true, 10, 4),
3     new Transport("Aviv", false, 9, 4),
4     new Transport("Aviv", true, 12, 4),
5     new Transport("Aviv", false, 12, 4),
6     new Transport("Aviv", true, 12, 6),
7 };
8
9 print(dayReport(arr, 4, true));    // 22
10 print(dayReport(arr, 4, false));  // 21
11
12 public static int dayReport(Transport[] arr, int day,
13                             boolean forward)
14 {
15     int sum = 0;
16     for (Transport tr: arr) {
17         if (tr.getDay() == day &&
18             tr.getToSchool() == forward)
19         {
20             sum += tr.getNum();
21         }
22     }
23     return sum;
24 }
```

(2) כתבו פעולה חיצונית ששמה moreForward בשפת Java או בשפת C#, המקבלת את המערך arr. הפעולה תדפיס את מספרי הימים בשבוע שבהם מספר התלמידים שנסעו ב"הסעות הלוך" היה גדול יותר ממספר התלמידים שנסעו ב"הסעות חזור". הערה: אפשר להיעזר בפעולה שכתבתם בתת-סעיף ב(1).

```
1 Transport[] arr = new Transport[] {
2     new Transport("Aviv", true, 10, 4),
3     new Transport("Aviv", false, 9, 4),
4     new Transport("Aviv", true, 12, 4),
5     new Transport("Aviv", false, 12, 4),
6     new Transport("Aviv", true, 12, 6),
7 };
8
9 moreForward(arr);    // 4, 6
10
11 public static void moreForward(Transport[] arr) {
12     for (int day = 1; day <= 6; day++) {
13         if (dayReport(arr, day, true) >
14             dayReport(arr, day, false))
15         {
16             System.out.println(day);
17         }
18     }
19 }
```

נתונה המחלקה **User** – משתמש ברשת חברתית, ולה שלוש תכונות:

- name – שם המשתמש מטיפוס מחרוזת (ייתכנו כמה אנשים שלהם אותו השם)
- id – מספר זהות מטיפוס שלם (מספר ייחודי)
- friends – מערך מטיפוס שלם, ובו מספרי הזהות של החברים של המשתמש (כל מספר מופיע פעם אחת בלבד). גודל המערך הוא בהתאם לכמות החברים של המשתמש.

הניחו שקיימות פעולות get/Set לתכונות המחלקה.

א. כתבו במחלקה **User** פעולה פנימית ששמה mutual בשפת Java או בשפת C#, המקבלת משתמש (User) אחר – other. הפעולה תחזירה את כמות החברים המשותפים של שני המשתמשים.

```
1 class User {
2     private String name;
3     private int id;
4     private int[] friends;
5 }
6
7 User[] users = new User[] {
8     new User("A", 111, new int[] {222, 333, 444}),
9     new User("B", 222, new int[] {333, 444}),
10    new User("C", 333, new int[] {111}),
11    new User("D", 444, new int[] {111}),
12 };
13
14 print(users[0].mutual(users[1]));    // 2
15 print(users[2].mutual(users[3]));    // 1
```

3. נתונה המחלקה **SocialNetwork** – רשת חברתית, ולה תכונה אחת:

• users – מערך מטיפוס **User**

המערך אינו ממוין לפי סדר כלשהו, כל משתמש מופיע בו פעם אחת בלבד, ואין בו ערכי null.

כתבו במחלקה **SocialNetwork** פעולה פנימית ששמה `exactOne` בשפת Java או `ExactOne` בשפת C#, המקבלת משתמש (**User**) – `other`, שאינו מופיע במערך. הפעולה מחזירה `true` אם קיים במערך משתמש אחד לפחות שלו ול-`other` יש בדיוק חבר אחד משותף. אחרת הפעולה מחזירה `false`.

```
1 class SocialNetwork {
2     private User[] users;
3 }
4
5 User[] users = new User[] {
6     new User("A", 111, new int[] {222, 333, 444}),
7     new User("B", 222, new int[] {333, 444}),
8     new User("C", 333, new int[] {111}),
9     new User("D", 444, new int[] {111}),
10 };
11
12 SocialNetwork sn = new SocialNetwork(users);
13 User e = new User("E", 555, new int[] {111});
14 User f = new User("F", 666, new int[] {333, 444});
15
16 print(sn.exactOne(e));    // true
17 print(sn.exactOne(f));    // false
```

א. כתבו במחלקה **User** פעולה פנימית ששמה `mutual` בשפת Java או `Mutual` בשפת C#, המקבלת משתמש (**User**) אחר – `other`, הפעולה מחזירה את כמות החברים המשותפים של שני המשתמשים.

```
1 class User {
2     private String name;
3     private int id;
4     private int[] friends;
5
6     // פונקציה עזר: מקבל מס' של משתמש אחר
7     // ומחזיר אמת אם הוא קיים בין החברים שלי
8     private boolean isFriend(int id) {
9         for (int i = 0; i < this.friends.length; i++) {
10             if (this.friends[i] == id) {
11                 return true;
12             }
13         }
14         return false;
15     }
16
17     public int mutual(User other) {
18         int count = 0;
19         for (int id: other.getFriends()) {
20             if (isFriend(id)) {
21                 count++;
22             }
23         }
24         return count;
25     }
26 }
```

מספר חיובי – `num` ייקרא "מספר מושלם" אם מתקיים התנאי שלפניכם:

מחברים את כל המחלקים החיוביים והשלמים של `num` (המספרים שאם מחלקים את `num` בהם מקבלים תוצאה ללא שארית), חוץ מ-`num`, והסכום שמתקבל שווה למספר המקורי.

דוגמה למספר מושלם: המספר 6, כי המחלקים שלו הם: 1, 2, 3, והסכום שלהם הוא 6.

דוגמה נוספת למספר מושלם: המספר 28, כי המחלקים שלו הם: 1, 2, 4, 7, 14, והסכום שלהם הוא 28.

דוגמה למספר שאינו מספר מושלם: המספר 8, כי המחלקים שלו הם: 1, 2, 4, והסכום שלהם הוא 7.

3. נתונה המחלקה **SocialNetwork** – רשת חברתית, ולה תכונה אחת:

• users – מערך מטיפוס **User**

המערך אינו ממוין לפי סדר כלשהו, כל משתמש מופיע בו פעם אחת בלבד, ואין בו ערכי null.

כתבו במחלקה **SocialNetwork** פעולה פנימית ששמה `exactOne` בשפת Java או `ExactOne` בשפת C#, המקבלת משתמש (**User**) – `other`, שאינו מופיע במערך. הפעולה מחזירה `true` אם קיים במערך משתמש אחד לפחות שלו ול-`other` יש בדיוק חבר אחד משותף. אחרת הפעולה מחזירה `false`.

```
1 class SocialNetwork {
2     private User[] users;
3
4     public boolean exactOne(User other) {
5         for (int i = 0; i < this.users.length; i++) {
6             if (this.users[i].mutual(other) == 1) {
7                 return true;
8             }
9         }
10         return false;
11     }
12 }
```

א. (1) כתבו פעולה חיצונית ששמה isPerfect בשפת Java או IsPerfect בשפת C#, המקבלת מספר שלם - num הגדול מ־2. הפעולה מתחזרת true אם הוא מספר מושלם, אחרת היא מחזירה false.
(2) כתבו פעולה חיצונית ששמה thePerfects בשפת Java או ThePerfects בשפת C#, המקבלת שני מספרים שלמים הגדולים מ־2 : low , high. הפעולה תדפיס את כל המספרים המושלמים שבין שני מספרים אלה (כולל).
הניחו ש־ low קטן מ־ high או שווה לו.
הערה: אפשר להיעזר בפעולה שכתבתם בתת־סעיף (1).

```
1 public static boolean isPerfect(int num) {
2     int sum = 0;
3     for (int i = 1; i < num; i++) {
4         if (num % i == 0) {
5             sum += i;
6         }
7     }
8     return sum == num;
9 }
10
11 public static void thePerfects(int low, int high) {
12     for (int num = low; num <= high; num++) {
13         if (isPerfect(num)) {
14             System.out.println(num);
15         }
16     }
17 }
```

יש השערה של א קינים מספר אי־זוגי שהוא מספר מושלם.
כתבו פעולה חיצונית ששמה noOdd בשפת Java או NoOdd בשפת C#, המתחזרת true אם אכן מבין כל המספרים 3 עד 999,999 (כולל) אין מספר מושלם שהוא אי־זוגי. אחרת היא מתחזרת false.

```
1 public static boolean noOdd() {
2     for (int num = 3; num <= 999999; num += 2) {
3         if (isPerfect(num)) {
4             return false;
5         }
6     }
7     return true;
8 }
```